

SQL Queries

Goal 1- Reduce Average Delivery time

1. Average time_taken_min:

```
select round(avg(time_taken_min),2) as average,  
count(*) as total_orders  
from fact_orders;
```

	average	total_orders
▶	26.29	45584

2. Delivery time by traffic density

```
select traffic_density,  
round(avg(time_taken_min),2) as averageT,  
count(*) as total_orders  
from fact_orders  
group by traffic_density  
order by averageT desc;
```

	traffic_density	averageT	total_orders
▶	Jam	31.18	14139
	High	27.24	4423
	Medium	26.70	10945
	Unknown	26.54	601
	Low	21.27	15476

3. Delivery time by weather condition

```
select weather_condition,  
round(avg(time_taken_min),2) as averageW,  
count(*) as total_orders  
from fact_orders  
group by weather_condition  
order by averageW desc;
```

	weather_condition	averageW	total_orders
	Fog	28.91	7653
	Unknown	26.55	616
	Windy	26.12	7422
	Sandstorms	25.88	7494
	Stormy	25.87	7584
	Sunny	21.86	7282

4. Festival day impact

```

select festival,
round(avg(time_taken_min),2) as averageF,
count(*) as total_orders
from fact_orders
group by festival
order by averageF desc;

```

	festival	averageF	total_orders
►	Yes	45.52	896
	No	25.98	44460
	Unknown	11.17	228

5. Distribution of Time_taken_min

With Quartiles as (

```

select
time_taken_min,
ntile(4) over(order by time_taken_Min) as qtile
from fact_orders)

```

```

select
qtile,
min(time_taken_min) as minT,
max(time_taken_min) as maxT,
count(*) as total_orders
from Quartiles
group by qtile
order by qtile;

```

	qtile	minT	manT	total_orders
▶	1	10	19	11396
	2	19	26	11396
	3	26	32	11396
	4	32	54	11396

6. Trend over Time

```
select order_date,
avg(time_taken_min) as avgO
from fact_orders
group by order_date
order by order_date;
```

order_date	avgO
2022-02-14	30.0611
2022-02-15	23.2942
2022-02-16	30.2950
2022-02-17	23.4952
2022-02-18	29.6690
2022-03-01	23.2263
2022-03-02	29.5079
2022-03-03	23.0548
2022-03-04	29.5673

RCA- Goal 1

1. Traffic density is a real driver — Jam adds ~10 min vs Low traffic (31% of all orders affected)
2. Festival days add ~75% to delivery time (45.5 vs 26.0 min) — small sample (896 orders) but large, consistent effect
3. Fog is surprisingly worse than storms/sandstorms for delivery time — a non-obvious weather finding worth highlighting
4. The slowest 25% of deliveries has 3x the spread of the fastest 25% — meaning the "bad" deliveries aren't consistently bad, they're unpredictable, which itself is a finding (points toward inconsistent handling of edge-case conditions, not a single fixable bottleneck)

Goal 2- Optimize Rider Performance

1. average delivery time per rider

```
SELECT
  delivery_person_id,
  ROUND(AVG(time_taken_min), 2) AS avg_delivery_time_min,
  COUNT(*) AS total_orders
FROM fact_orders
GROUP BY delivery_person_id
ORDER BY avg_delivery_time_min DESC
LIMIT 20;
```

	delivery_person_id	avg_delivery_time_min	total_orders
▶	ALHRES12DEL01	35.00	12
	KOLRES19DEL02	33.83	12
	AGRRES17DEL01	33.67	12
	KNPRES14DEL01	32.86	14
	GOARES08DEL03	32.40	10
	ALHRES010DEL01	32.21	14
	KOLRES03DEL03	32.18	11
	ALHRES18DEL02	32.15	13
	KNPRES06DEL01	32.14	14
	DEHRES09DEL02	32.08	12
	KOLRES02DEL02	31.91	11
	ALHRES03DEL01	31.75	12
	LUDHRES01DEL03	31.75	8
	AURGRES14DEL02	31.69	13
	KOLRES09DEL03	31.50	6
	ALHRES17DEL03	31.50	14
	BHPRES03DEL02	31.31	13
	BHPRES14DEL03	31.20	10
	DEHRES13DEL03	31.17	12
	KNPRES08DEL02	31.17	12

2. Rank riders within their own city

```
SELECT
  fo.delivery_person_id,
  dp.city_code,
  ROUND(AVG(fo.time_taken_min), 2) AS avg_delivery_time_min,
  COUNT(*) AS total_orders,
  RANK() OVER (PARTITION BY dp.city_code ORDER BY AVG(fo.time_taken_min)
DESC) AS slowest_rank_in_city
```

```

FROM fact_orders fo
JOIN dim_delivery_person dp ON fo.delivery_person_id = dp.delivery_person_id
GROUP BY fo.delivery_person_id, dp.city_code
HAVING COUNT(*) >= 10
ORDER BY dp.city_code, slowest_rank_in_city;

```

	delivery_person_id	city_code	avg_delivery_time_min	total_orders	slowest_rank_in_city
	AGRRES17DEL01	AGR	33.67	12	1
	AGRRES02DEL01	AGR	30.82	11	2
	AGRRES08DEL01	AGR	30.27	11	3
	AGRRES07DEL01	AGR	30.18	11	4
	AGRRES08DEL02	AGR	29.93	15	5
	AGRRES13DEL02	AGR	29.50	14	6
	AGRRES16DEL02	AGR	29.07	15	7
▶	AGRRES09DEL03	AGR	29.00	10	8
	AGRRES04DEL02	AGR	28.92	12	9
	AGRRES010DEL02	AGR	28.80	15	10
	AGRRES03DEL01	AGR	28.77	13	11
	AGRRES13DEL01	AGR	28.71	14	12
	AGRRES19DEL03	AGR	28.67	12	13
	AGRRES13DEL03	AGR	28.38	13	14
	AGRRES20DEL01	AGR	28.29	14	15
	AGRRES07DEL03	AGR	28.27	15	16
	AGRRES14DEL01	AGR	28.25	12	17
	AGRRES12DEL02	AGR	27.93	15	18
	AGRRES09DEL01	AGR	27.85	13	19
	AGRRES16DEL01	AGR	27.77	13	20
	AGRRES19DEL02	AGR	27.73	11	21

	delivery_person_id	city_code	avg_delivery_time_min	total_orders	slowest_rank_in_city
	ALHRES16DEL03	ALH	24.50	10	48
	ALHRES11DEL01	ALH	24.33	15	49
	ALHRES02DEL02	ALH	24.20	10	50
	ALHRES06DEL02	ALH	24.00	13	51
	ALHRES05DEL01	ALH	23.64	11	52
	ALHRES05DEL02	ALH	23.63	16	53
	ALHRES06DEL01	ALH	23.58	12	54
	ALHRES04DEL02	ALH	22.92	13	55
	AURGRES14DEL02	AURG	31.69	13	1
	AURGRES010DEL03	AURG	30.73	11	2
	AURGRES09DEL03	AURG	29.40	10	3
	AURGRES05DEL02	AURG	29.27	15	4
	AURGRES010DEL02	AURG	28.82	11	5
	AURGRES02DEL01	AURG	28.79	14	6
	AURGRES17DEL02	AURG	28.71	14	7
	AURGRES16DEL01	AURG	28.67	15	8
	AURGRES06DEL01	AURG	28.55	11	9
	AURGRES010DEL01	AURG	28.50	12	10
	AURGRES03DEL01	AURG	28.46	13	11
▶	AURGRES15DEL02	AURG	28.46	13	11
	AURGRES13DEL01	AURG	28.00	12	13

3. Consistency — standard deviation per rider

```
SELECT
    delivery_person_id,
    ROUND(AVG(time_taken_min), 2) AS avg_delivery_time_min,
    ROUND(STDDEV(time_taken_min), 2) AS delivery_time_stddev,
    COUNT(*) AS total_orders
FROM fact_orders
GROUP BY delivery_person_id
HAVING COUNT(*) >= 10
ORDER BY delivery_time_stddev DESC
LIMIT 20;
```

	delivery_person_id	avg_delivery_time_min	delivery_time_stddev	total_orders
▶	KOLRES08DEL02	28.60	14.19	10
	DEHRES07DEL02	28.08	13.41	12
	KNPRES03DEL02	26.10	13.35	10
	LUDHRES05DEL01	29.08	13.26	12
	KNPRES19DEL01	26.86	13.12	14
	KOCRES20DEL02	26.50	13.03	10
	AGRRES03DEL01	28.77	13	13
	AGRRES17DEL03	26.36	13	14
	ALHRES03DEL01	31.75	12.77	12
	AGRRES02DEL01	30.82	12.63	11
	AGRRES09DEL03	29.00	12.59	10
	KNPRES05DEL03	29.09	12.59	11
	ALHRES12DEL01	35.00	12.46	12
	LUDHRES18DEL03	30.31	12.39	13
	BHPRES02DEL02	25.00	12.35	10
	GOARES18DEL01	26.21	12.31	14
	BHPRES13DEL02	31.17	12.2	12
	AGRRES19DEL02	27.73	12.16	11
	DEHRES08DEL03	27.82	12.12	11
	ALHRES07DEL02	29.46	12.03	13

4. Best vs worst rider gap, per city

```
WITH rider_avg AS (
    SELECT
        fo.delivery_person_id,
        dp.city_code,
        AVG(fo.time_taken_min) AS avg_time,
        COUNT(*) AS total_orders
    FROM fact_orders fo
```

```

JOIN dim_delivery_person dp ON fo.delivery_person_id =
dp.delivery_person_id
GROUP BY fo.delivery_person_id, dp.city_code
HAVING COUNT(*) >= 10
)
SELECT
city_code,
ROUND(MIN(avg_time), 2) AS best_rider_avg,
ROUND(MAX(avg_time), 2) AS worst_rider_avg,
ROUND(MAX(avg_time) - MIN(avg_time), 2) AS
performance_gap_min
FROM rider_avg
GROUP BY city_code
ORDER BY performance_gap_min DESC;

```

	city_code	best_rider_avg	worst_rider_avg	performance_gap_min
►	KOL	19.82	33.83	14.02
	KNP	20.00	32.86	12.86
	DEH	19.60	32.08	12.48
	AGR	21.25	33.67	12.42
	ALH	22.92	35.00	12.08
	LUDH	20.20	30.90	10.70
	GOA	22.00	32.40	10.40
	AURG	21.50	31.69	10.19
	KOC	21.23	30.38	9.15
	BHP	22.36	31.31	8.94
	COIMB	21.88	29.09	7.20
	CHEN	23.59	30.57	6.98
	PUNE	22.60	29.41	6.82
	VAD	22.78	29.24	6.46
	RANCHI	22.87	29.29	6.42
	MUM	22.80	28.41	5.61
	HYD	23.30	28.81	5.51
	MYS	23.28	28.45	5.16
	BANG	23.07	28.20	5.14
	SUR	23.81	28.80	4.99
	INDO	23.75	28.48	4.73
	JAP	24.42	29.15	4.72

5. Rating vs. speed correlation

```

SELECT
CASE
WHEN delivery_person_rating >= 4.5 THEN '4.5+'
WHEN delivery_person_rating >= 4.0 THEN '4.0-4.49'
WHEN delivery_person_rating >= 3.5 THEN '3.5-3.99'
WHEN delivery_person_rating > 0 THEN 'Below 3.5'
ELSE 'Unknown/Missing'

```

```

END AS rating_band,
ROUND(AVG(time_taken_min), 2) AS avg_delivery_time_min,
COUNT(*) AS order_count
FROM fact_orders
GROUP BY rating_band
ORDER BY rating_band DESC;

```

	rating_band	avg_delivery_time_min	order_count
▶	Unknown/Missing	26.42	1908
	Below 3.5	35.07	261
	4.5+	24.31	35615
	4.0-4.49	34.65	6694
	3.5-3.99	37.25	1106

RCA- Goal 2

1. City tier drives rider consistency more than the individual rider does — the gap between a city's best and worst rider ranges from ~14 min in smaller cities (Kolkata, Kanpur, Dehradun, Agra, Allahabad) down to ~4.7–5.6 min in large metros (Jaipur, Surat, Bangalore, Mysore, Hyderabad, Mumbai) — roughly a 3x difference in consistency between city tiers.
2. Low order volume, not poor skill, likely explains the "worst" riders — both the slowest-riders list and the most-erratic-riders list are dominated by riders with only 10–16 total orders, versus 40–67 orders for typical metro riders. Small samples naturally swing more extreme, so this is a sample-size effect rather than confirmed evidence of individual underperformance.
3. Smaller cities have fewer orders per rider overall — this is the likely root cause behind Finding 1: fewer orders per rider in smaller cities means less operational data to stabilize their average, producing the appearance of inconsistency where it may really be an under-resourcing/demand issue.
4. Rating and speed are related, but not linearly — the top rating band (4.5+, on a large 35,615-order sample) is clearly fastest at 24.31 min, but the relationship breaks down below that: 3.5–3.99 is actually *slower* than Below 3.5 (37.25 vs 35.07 min), based on much smaller samples (1,106 and 261 orders) — flagged as directional, not confirmed, pending a larger sample.

Goal 3- Optimize Resource Allocation Across Cities

1. Order volume per city

```
SELECT
    dp.city_code,
    COUNT(*) AS total_orders
FROM fact_orders fo
JOIN dim_delivery_person dp ON fo.delivery_person_id = dp.delivery_person_id
GROUP BY dp.city_code
ORDER BY total_orders DESC;
```

	city_code	total_orders
▶	JAP	3443
	RANCHI	3228
	BANG	3193
	SUR	3187
	HYD	3179
	MUM	3173
	MYS	3170
	COIMB	3169
	VAD	3166
	INDO	3158
	CHEN	3144
	PUNE	3132
	AGR	763
	LUDH	75 3179
	ALH	740
	KNP	740
	DEH	737
	GOA	709
	AURG	703
	KOC	701
	KOL	700
	BHP	691

2. Rider headcount per city

```
SELECT
    city_code,
    COUNT(DISTINCT delivery_person_id) AS total_riders
FROM dim_delivery_person
GROUP BY city_code
ORDER BY total_riders DESC;
```

	city_code	total_riders
▶	AGR	60
	ALH	60
	AURG	60
	BANG	60
	BHP	60
	CHEN	60
	COIMB	60
	DEH	60
	GOA	60
	HYD	60
	INDO	60
	JAP	60
	KNP	60
	KOC	60
	KOL	60
	LUDH	60
	MUM	60
	MYS	60
	PUNE	60
	RANCHI	60
	SUR	60
	VAD	60

3. The real test — orders-per-rider ratio

```

WITH city_orders AS (
    SELECT dp.city_code, COUNT(*) AS total_orders
    FROM fact_orders fo
    JOIN dim_delivery_person dp ON fo.delivery_person_id = dp.delivery_person_id
    GROUP BY dp.city_code
),
city_riders AS (
    SELECT city_code, COUNT(DISTINCT delivery_person_id) AS total_riders
    FROM dim_delivery_person
    GROUP BY city_code
)
SELECT
    co.city_code,
    co.total_orders,
    cr.total_riders,
    ROUND(co.total_orders / cr.total_riders, 1) AS orders_per_rider
FROM city_orders co
JOIN city_riders cr ON co.city_code = cr.city_code
ORDER BY orders_per_rider DESC;

```

	city_code	total_orders	total_riders	orders_per_rider
	BANG	3193	60	53.2
	SUR	3187	60	53.1
	HYD	3179	60	53.0
	MUM	3173	60	52.9
	COIMB	3169	60	52.8
	MYS	3170	60	52.8
	VAD	3166	60	52.8
	INDO	3158	60	52.6
	CHEN	3144	60	52.4
	PUNE	3132	60	52.2
	AGR	763	60	12.7
	LUDH	758	60	12.6
	ALH	740	60	12.3
	DEH	737	60	12.3
	KNP	740	60	12.3
	GOA	709	60	11.8
	AURG	703	60	11.7
	KOC	701	60	11.7
	KOL	700	60	11.7
	BHP	691	60	11.5

4. Does higher orders-per-rider correlate with slower delivery?

```

WITH city_orders AS (
    SELECT dp.city_code, COUNT(*) AS total_orders,
    ROUND(AVG(fo.time_taken_min),2) AS avg_delivery_time
    FROM fact_orders fo
    JOIN dim_delivery_person dp ON fo.delivery_person_id = dp.delivery_person_id
    GROUP BY dp.city_code
),
city_riders AS (
    SELECT city_code, COUNT(DISTINCT delivery_person_id) AS total_riders
    FROM dim_delivery_person
    GROUP BY city_code
)
SELECT
    co.city_code,
    co.total_orders,
    cr.total_riders,
    ROUND(co.total_orders / cr.total_riders, 1) AS orders_per_rider,
    co.avg_delivery_time
FROM city_orders co
JOIN city_riders cr ON co.city_code = cr.city_code
ORDER BY orders_per_rider DESC;

```

	city_code	total_orders	total_riders	orders_per_rider	avg_delivery_time
▶	JAP	3443	60	57.4	26.69
	RANCHI	3228	60	53.8	26.24
	BANG	3193	60	53.2	26.05
	SUR	3187	60	53.1	26.08
	HYD	3179	60	53.0	26.43
	MUM	3173	60	52.9	26.05
	COIMB	3169	60	52.8	26.29
	MYS	3170	60	52.8	26.00
	VAD	3166	60	52.8	26.30
	INDO	3158	60	52.6	26.12
	CHEN	3144	60	52.4	26.36
	PUNE	3132	60	52.2	26.34
	AGR	763	60	12.7	26.64
	LUDH	758	60	12.6	26.28
	ALH	740	60	12.3	27.03
	DEH	737	60	12.3	26.68
	KNP	740	60	12.3	26.35
	GOA	709	60	11.8	26.20
	AURG	703	60	11.7	26.65
	KOC	701	60	11.7	26.35
	KOL	700	60	11.7	26.36
	BHP	691	60	11.5	26.80

5. Area type (Metropolitan/Urban/Semi-Urban) resource pattern

SELECT

fo.area_type,

COUNT(*) AS total_orders,

ROUND(AVG(fo.time_taken_min), 2) AS avg_delivery_time,

COUNT(DISTINCT fo.delivery_person_id) AS unique_riders

FROM fact_orders fo

GROUP BY fo.area_type

ORDER BY avg_delivery_time DESC;

	city	total_orders	avg_delivery_time	unique_riders
▶	Semi-Urban	164	49.73	156
	Metropolitan	34087	27.31	1320
	Urban	10133	22.98	1285
	Unknown	1200	22.06	688

RCA- goal 3

1. Massive order-volume imbalance exists despite identical rider headcount across every city — every city has exactly 60 riders, but large cities (Bangalore, Surat, Hyderabad, Mumbai, etc.) handle 3,100–3,400 orders, while smaller cities (Agra, Ludhiana, Allahabad, Dehradun, etc.) handle only ~700–760 orders — a ~4.3x demand gap on an identical rider base. *(Caveat worth noting: the flat "60 riders per*

city" figure looks like a structural pattern from how rider IDs were generated in this dataset — 20 restaurants × 3 riders — rather than a deliberate real-world staffing decision. The demand imbalance itself is real; the "equal headcount" should be read as a fixed dataset structure, not evidence Zomato intentionally staffed every city equally.)

2. This resourcing imbalance does NOT translate into slower average delivery times — average delivery time stays remarkably flat (26.0–27.0 min) across every city regardless of load, from JAP's heaviest load (57.4 orders/rider, 26.69 min) down to BHP's lightest (11.5 orders/rider, 26.80 min). This is an important finding because it actually disproves the theory from Goal 2 — low order volume per rider does not appear to cause genuinely slower service.
3. This reframes Goal 2's "consistency gap" finding — since average performance holds steady regardless of city load, the earlier finding (small cities showing 3x wider best/worst-rider performance gaps) is better explained by statistical noise from small sample sizes, not real service degradation in under-resourced cities. This is a useful, honest correction to build into your RCA narrative — it shows a hypothesis being tested and refined with evidence, not just confirmed blindly.
4. Metropolitan area type carries ~75% of total order volume (34,087 of 45,584 orders) at a solid 27.31 min average — this is the operational backbone of the business, and any resourcing or performance recommendation should be anchored here first.
5. Semi-Urban and "Unknown" area types are unreliable outliers, not confirmed patterns — Semi-Urban shows a dramatic 49.73 min average, but on only 164 orders with 156 unique riders (~1 order per rider) — far too thin a sample to draw a real conclusion. Flag as "needs more data" rather than a finding to act on

Goal 4- Improve Customer Experience

1. Set the SLA threshold and see overall breach rate

SELECT

COUNT(*) AS total_orders,

SUM(CASE WHEN time_taken_min > 40 THEN 1 ELSE 0 END) AS breached_orders,

ROUND(SUM(CASE WHEN time_taken_min > 40 THEN 1 ELSE 0 END) * 100.0 /

COUNT(*), 2) AS breach_rate_pct

FROM fact_orders;

	total_orders	breached_orders	breach_rate_pct
▶	45584	4037	8.86

2. SLA breach rate by city — where is customer experience worst?

```

SELECT
    dp.city_code,
    COUNT(*) AS total_orders,
    SUM(CASE WHEN fo.time_taken_min > 40 THEN 1 ELSE 0 END) AS
breached_orders,
    ROUND(SUM(CASE WHEN fo.time_taken_min > 40 THEN 1 ELSE 0 END) * 100.0 /
COUNT(*), 2) AS breach_rate_pct
FROM fact_orders fo
JOIN dim_delivery_person dp ON fo.delivery_person_id = dp.delivery_person_id
GROUP BY dp.city_code
ORDER BY breach_rate_pct DESC;

```

	city_code	total_orders	breached_orders	breach_rate_pct
▶	KOC	701	71	10.13
	DEH	737	74	10.04
	JAP	3443	344	9.99
	BHP	691	67	9.70
	VAD	3166	303	9.57
	AURG	703	67	9.53
	ALH	740	70	9.46
	AGR	763	72	9.44
	HYD	3179	295	9.28
	KNP	740	68	9.19
	CHEN	3144	288	9.16
	PUNE	3132	284	9.07
	COIMB	3169	281	8.87
	RANCHI	3228	286	8.86
	INDO	3158	268	8.49
	MYS	3170	265	8.36
	MUM	3173	261	8.23
	KOL	700	57	8.14
	LUDH	758	60	7.92
	GOA	709	56	7.90
	BANG	3193	251	7.86
	SUR	3187	249	7.81

3. SLA breach rate by traffic + festival combined

```

SELECT
    traffic_density,
    festival,
    COUNT(*) AS total_orders,
    ROUND(SUM(CASE WHEN time_taken_min > 40 THEN 1 ELSE 0 END) * 100.0 /
COUNT(*), 2) AS breach_rate_pct

```

```

FROM fact_orders
GROUP BY traffic_density, festival
ORDER BY breach_rate_pct DESC;

```

	traffic_density	festival	total_orders	breach_rate_pct
►	Unknown	Yes	17	100.00
	Jam	Yes	630	92.06
	High	Yes	69	91.30
	Medium	Yes	130	82.31
	Low	Yes	50	78.00
	Jam	No	13469	16.71
	Unknown	No	581	6.37
	Medium	No	10776	5.12
	High	No	4336	5.00
	Low	No	15298	1.14
	Medium	Unknown	39	0.00
	Low	Unknown	128	0.00
	High	Unknown	18	0.00
	Jam	Unknown	40	0.00
	Unknown	Unknown	3	0.00

4. Delivery time distribution

```

SELECT
CASE
  WHEN time_taken_min <= 20 THEN '1. Excellent (≤20 min)'
  WHEN time_taken_min <= 30 THEN '2. Good (21-30 min)'
  WHEN time_taken_min <= 40 THEN '3. Acceptable (31-40 min)'
  ELSE '4. Poor (40+ min)'
END AS experience_band,
COUNT(*) AS order_count,
ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM fact_orders), 2) AS
pct_of_total
FROM fact_orders
GROUP BY experience_band
ORDER BY experience_band;

```

	experience_band	order_count	pct_of_total
►	1. Excellent (≤20 min)	14149	31.04
	2. Good (21-30 min)	17833	39.12
	3. Acceptable (31-40 min)	9565	20.98
	4. Poor (40+ min)	4037	8.86

5. Does a bad experience correlate with a rider getting a lower rating on that specific order?

```
SELECT
CASE
  WHEN time_taken_min <= 30 THEN 'On-time (≤30 min)'
  ELSE 'Late (30+ min)'
END AS delivery_status,
ROUND(AVG(NULLIF(delivery_person_rating, 0)), 2) AS avg_rating,
COUNT(*) AS order_count
FROM fact_orders
GROUP BY delivery_status;
```

	delivery_status	avg_rating	order_count
▶	On-time (≤30 min)	4.73	31982
	Late (30+ min)	4.41	13602

RCA-goal 4

1. Overall SLA breach rate is 8.86% — roughly 1 in 11 orders exceeds the 40-minute threshold. On its own this looks manageable, but the next findings show it's heavily concentrated, not evenly spread.
2. Festival days are the single biggest driver of poor customer experience by a huge margin — breach rate jumps from ~1–17% on normal days (depending on traffic) to 78–100% on festival days, regardless of traffic level. Even *Low* traffic on a festival day breaches 78% of the time — worse than *Jam* traffic on a normal day (16.71%). This confirms and sharpens the Goal 1 festival finding: festival status overrides traffic as the dominant factor in customer experience, not the other way around.
3. Jam + Festival is the single worst combination — 92.06% breach rate (630 orders). This is your clearest "compounding factor" finding across all four goals so far — the two worst individual conditions from Goal 1 (Jam, Festival) combine multiplicatively, not just additively.
4. City-level breach rate is fairly uniform and doesn't track city size — breach rates cluster in a narrow 7.8–10.1% band across both large metros (Jaipur 9.99%, Hyderabad 9.28%) and small cities (Kochi 10.13%, Dehradun 10.04%). This reinforces the Goal 3 finding: city size/resourcing doesn't meaningfully predict service quality — condition-based factors (festival, traffic) matter far more than which city an order is in.
5. The overall distribution is healthy apart from the tail — 70% of orders fall in "Excellent" or "Good" bands (≤30 min), and only 8.86% are "Poor." This means the customer experience problem isn't systemic — it's concentrated in specific,

identifiable conditions (festival days, especially combined with traffic), which is actually a *more actionable* finding than if the whole system ran slow.

Goal 5-Understand Operational Bottlenecks

1. Order preparation delay

```
SELECT
ROUND(AVG(TIMESTAMPDIFF(
MINUTE,
STR_TO_DATE(time_ordered,'%H:%i'),
STR_TO_DATE(time_order_picked,'%H:%i')
)),2) AS avg_prep_time,
MAX(TIMESTAMPDIFF(
MINUTE,
STR_TO_DATE(time_ordered,'%H:%i'),
STR_TO_DATE(time_order_picked,'%H:%i')
)) AS max_prep_time
FROM fact_orders;
```

	avg_prep_time	max_prep_time
▶	23.11	1435

2. Preparation delay by order type

```
SELECT
order_type,
ROUND(AVG(
TIMESTAMPDIFF(
MINUTE,
STR_TO_DATE(time_ordered,'%H:%i'),
STR_TO_DATE(time_order_picked,'%H:%i')
)
),2) AS avg_prep_time,
COUNT(*) AS total_orders
FROM fact_orders
GROUP BY order_type
ORDER BY avg_prep_time DESC;
```

	order_type	avg_prep_time	total_orders
▶	Snack	19.38	11530
	Meal	23.38	11456
	Drinks	27.42	11321
	Buffet	22.31	11277

3. Preparation delay by traffic density

```

SELECT
traffic_density,
ROUND(AVG(
TIMESTAMPDIFF(
MINUTE,
STR_TO_DATE(time_ordered,'%H:%i'),
STR_TO_DATE(time_order_picked,'%H:%i')
)
),2) AS avg_prep_time,
COUNT(*) AS total_orders
FROM fact_orders
GROUP BY traffic_density
ORDER BY avg_prep_time DESC;

```

	traffic_density	avg_prep_time	total_orders
▶	Unknown	1041.53	601
	Jam	39.86	14139
	Medium	36.22	10945
	High	27.99	4423
	Low	-42.42	15476

4. Vehicle condition vs delivery time

```

SELECT
vehicle_condition,
ROUND(AVG(time_taken_min),2) AS avg_delivery_time,
COUNT(*) AS total_orders
FROM fact_orders
GROUP BY vehicle_condition
ORDER BY vehicle_condition;

```

	vehide_condition	avg_delivery_time	total_orders
▶	0	30.07	15005
	1	24.35	15028
	2	24.45	15031
	3	26.49	520

5. Multiple deliveries impact

SELECT multiple_deliveries, ROUND(AVG(time_taken_min),2) AS avg_delivery_time, COUNT(*) AS total_orders FROM fact_orders WHERE multiple_deliveries >= 0 GROUP BY multiple_deliveries ORDER BY multiple_deliveries;

	multiple_deliveries	avg_delivery_time	total_orders
▶	0	22.88	14094
	1	26.85	28151
	2	40.45	1985
	3	47.82	361

6. Combined operational bottlenecks

SELECT
multiple_deliveries,
vehicle_condition,
ROUND(AVG(time_taken_min),2) AS avg_delivery_time,
COUNT(*) AS total_orders
FROM fact_orders
WHERE multiple_deliveries >= 0
GROUP BY multiple_deliveries, vehicle_condition
HAVING COUNT(*) >= 30
ORDER BY avg_delivery_time DESC;

3	0	48.42	280
3	2	45.47	32
3	1	45.44	43
2	0	41.89	982
2	2	39.06	505
2	1	39.01	473
1	0	29.72	9542
0	0	26.93	3907
1	3	26.68	322
1	2	25.42	9124
1	1	25.31	9163
0	3	23.17	158
0	2	21.33	5032
0	1	21.25	4997

RCA-goal 5

1. Multiple deliveries are the strongest operational bottleneck identified in the dataset — average delivery time increases steadily from 22.88 min for single deliveries to 26.85 min for one additional delivery, 40.45 min for two deliveries, and 47.82 min for three deliveries. This clear upward trend indicates that delivery batching significantly slows fulfillment and is the largest controllable operational factor observed.
2. Vehicle condition has a measurable but smaller impact than delivery batching — riders using vehicles with condition 0 average 30.07 min, compared with approximately 24–26 min for vehicles in better condition. While fleet condition affects delivery speed, its impact is considerably smaller than the effect of handling multiple deliveries simultaneously.
3. The worst operational performance occurs when bottlenecks compound rather than occur independently — riders handling three simultaneous deliveries with vehicle condition 0 record the highest average delivery time (48.42 min), while the same workload with better vehicle conditions remains above 45 min. This suggests operational delays are amplified when multiple inefficiencies occur together instead of acting in isolation.
4. Preparation-time analysis reveals data quality limitations rather than operational insights — negative preparation times under Low traffic and extremely large preparation times (up to 1,435 minutes) indicate timestamp inconsistencies within the dataset. These fields are unsuitable for operational decision-making without additional data cleaning and should be treated as a data-quality issue rather than evidence of restaurant performance.
5. Overall, operational bottlenecks are driven primarily by workload management rather than infrastructure constraints — unlike earlier goals where traffic, festivals, and environmental conditions explained delivery delays, this analysis shows that internal operational decisions—particularly the number of deliveries assigned to a rider—have the strongest and most consistent impact on delivery performance. This

makes delivery allocation policies the most actionable area for operational improvement.